HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

Course: Object-Oriented Programming Using C++
Assignment Title: Hotel Accommodation and Reservation Management Using C++

TABLE OF CONTENTS

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

Hotel reservation management involves handling guests, rooms, reservations, hotel services and billing. Managing these operations manually can result in incorrect room allocation, duplicate reservations, billing errors and difficulty in maintaining guest information.

The proposed Hotel Accommodation and Reservation Management System is a menu-driven C++ application developed using Object-Oriented Programming principles. The system enables hotel staff to register guests, display available rooms, make and cancel reservations, add hotel services, calculate bills and generate reservation details.

The application uses different room and service categories through inheritance. Abstract classes, virtual functions, pointers and runtime polymorphism are incorporated to make the system modular, reusable and extensible.

1.2 Problem Formulation

The main purpose of the proposed system is to provide a simple and organized way to manage hotel accommodation and reservation activities. The system allows hotel staff to register and manage guest information, including basic details required for making a reservation. It also maintains different categories of hotel rooms, such as Standard, Deluxe, and Suite rooms, and displays their current availability so that users can select a suitable room.

The system supports creating and cancelling reservations based on room availability. Before confirming a reservation, the system checks whether the selected room is already occupied, thereby helping to prevent duplicate or overlapping bookings. Guests can also select additional hotel services, such as food or spa services, which are added to their reservation and included in the final bill. The system then calculates the total customer bill by combining the room charges with the charges for selected services.

Another objective is to generate and display complete reservation details, including guest information, room details, reservation information, and billing details. From a programming perspective, the system is designed to demonstrate important C++ concepts such as inheritance and runtime polymorphism by using suitable base and derived classes for different room and service types. Finally, the project also focuses on debugging and error handling, where programming errors such as incorrect calculations, invalid inputs, inheritance issues, pointer errors, and room-booking conflicts are identified and corrected to improve the reliability of the system.

2. OBJECTIVES AND EXPECTED OUTCOMES

2.1 Objectives

The project also aims to make use of abstract classes and pure virtual functions to define common operations while allowing derived classes to provide their own implementations. Multiple and hierarchical inheritance are included where they are useful for representing relationships between different components of the system. Virtual base classes are used to avoid duplication of common data when multiple inheritance is involved.

Another important objective is to demonstrate runtime polymorphism using base-class pointers, so that different room or service objects can be handled through a common interface. The system is also designed to manage guest reservations efficiently, including room allocation and cancellation, while ensuring that the charges for rooms and additional services are calculated accurately. Finally, the project focuses on the identification and correction of programming errors through debugging, helping improve the correctness, reliability, and overall performance of the application.

2.2 Expected Outcomes

The system successfully registers guests by collecting and storing their required personal and booking details. It displays the list of available rooms along with relevant room information, allowing guests to select a suitable room and create valid reservations. The system also verifies room availability and automatically rejects reservation requests for rooms that are already occupied or reserved. Guests can cancel their existing reservations when required, and the system supports the addition of extra services such as food, laundry, room service, or other hotel facilities. Based on the selected room, duration of stay, and additional services, the system calculates the final bill accurately. It also provides complete reservation information, including guest details, room details, reservation status, services selected, and total cost. The system demonstrates the concept of dynamic polymorphism through the use of inheritance and virtual functions, enabling different objects to perform operations according to their actual runtime types. Furthermore, appropriate validation is implemented to handle invalid inputs, incorrect reservation details, unavailable rooms, and runtime errors, ensuring that the system operates reliably and provides meaningful error messages to the user.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

The system should provide facilities for guest registration, room listing, room availability checking, room reservation, reservation cancellation, hotel service selection, bill calculation, reservation details, and menu-based navigation. These features allow the user to register guests, check the status of rooms, make or cancel reservations, select additional hotel services, calculate the total bill, and view complete reservation information through a simple menu-driven interface.

3.2 Non-Functional Requirements

The system should be easy to use, modular, maintainable, reusable, efficient, error-resistant, and extensible. It should have a simple structure that allows different modules to work independently while making it easier to modify or add new features in the future. The application should also provide reliable results, handle common errors properly, and remain efficient during normal usage.

3.3 Constraints

The application is developed using C++ and is designed to operate through a console-based interface. The system maintains data only during program execution, so the information will not be available after the program is closed. An external database is not required for the current implementation. In addition, only guests who have been registered in the system are allowed to make reservations.**3.4 Assumptions**

The system assumes that each reservation is associated with one guest and that a room cannot be reserved by more than one guest at the same time. The room charge is calculated based on the selected room category, while any additional hotel services are charged separately. When a reservation is cancelled, the corresponding room is considered available again and can be reserved by another guest.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE

C++ Concept	Application

Classes and Objects	Guest, Room, Reservation and Service
Encapsulation	Data members and member functions
Inheritance	Different room and service categories
Abstract Class	Base Room and Service classes
Pure Virtual Function	Room and service charge calculation
Virtual Function	Runtime polymorphism
Pointers	Dynamic object management
Runtime Polymorphism	Different room types through base pointer
Virtual Base Class	Avoiding duplicate base-class instances
Function Overriding	Room-specific charge calculation
Constructors	Object initialization
Destructors	Resource cleanup
Exception Handling	Invalid operations/input handling

5. PROPOSED SOLUTION AND METHODOLOGY

The proposed system follows a modular object-oriented approach. The application is divided into Guest Management, Room Management, Reservation Management, Hotel Services, Billing, and Debugging and Validation modules.

5.1 Guest Management

Responsible for guest registration, guest identification and guest information management.

5.2 Room Management

Responsible for maintaining room details, displaying room categories, checking availability and calculating room charges.

5.3 Reservation Management

Responsible for creating and cancelling reservations, assigning rooms to guests and maintaining reservation information.

5.4 Hotel Services

Responsible for food, laundry, spa and other additional services.

5.5 Billing

Responsible for calculating room charges, service charges and generating the final bill.

5.6 Debugging and Validation

Responsible for detecting invalid input, preventing duplicate reservations, handling invalid room numbers and validating reservation operations.

6. SYSTEM DESIGN

6.1 Class Design

The system consists of Guest, Staff, Reservation, Hotel, Room and Service classes. Room and Service are designed as abstract bases, with specialized derived classes for different categories.

Class hierarchy:

Person → Guest / Staff → Reservation → Hotel → Room (Abstract) → StandardRoom / DeluxeRoom / SuiteRoom

Service (Abstract) → FoodService / SpaService

6.2 Runtime Polymorphism

A base-class pointer can refer to different derived room objects. When a virtual function is called through the pointer, the appropriate derived-class implementation is selected at runtime.

6.3 Virtual Base Class

Virtual inheritance can be used where two inheritance paths lead to the same base class. This prevents multiple copies of the common base-class portion from being created.

7. ALGORITHM AND PSEUDOCODE

7.1 Main Algorithm

Step: Start the application.
Step: Display the main menu.
Step: Accept the user's choice.
Step: If Guest Registration is selected, collect guest details.
Step: If Room Availability is selected, display available rooms.
Step: If Reservation is selected, verify guest and room availability.
Step: Create the reservation if the room is available.
Step: If Services is selected, display available services.
Step: Add selected services to the reservation.
Step: Calculate room and service charges.
Step: If cancellation is selected, cancel the reservation and release the room.
Step: Display reservation details when requested.
Step: Continue until the Exit option is selected.
Step: Terminate the program.

7.2 Pseudocode

```
START

Initialize hotel system

REPEAT
  Display menu
  Read choice

  IF choice = 1
    Register guest
  ELSE IF choice = 2
    Display available rooms
  ELSE IF choice = 3
    Read guest ID
    Read room number
    IF room is available
      Create reservation
      Mark room as occupied
```

```
        ELSE
            Display "Room not available"
        END IF
    ELSE IF choice = 4
        Read reservation ID
        Cancel reservation
        Release room
    ELSE IF choice = 5
        Display hotel services
        Add selected service
    ELSE IF choice = 6
        Calculate room charge
        Calculate service charge
        Display total bill
    ELSE IF choice = 7
        Display reservation details
    ELSE IF choice = 8
        EXIT
    ELSE
        Display "Invalid choice"
    END IF
UNTIL choice = 8

STOP
```

8. FLOWCHART

START → Initialize System → Display Menu → Read User Choice → Guest Registration / Room Availability / Reservation → Service Selection → Calculate Bill → Display Reservation → Exit Selected? → STOP

9. IMPLEMENTATION

The implementation is carried out using C++ classes and object-oriented principles. The major implementation components include abstract base classes, derived room classes, virtual functions, pointers and dynamic object management.

9.1 Base Abstract Class

```
class Room {
public:
    virtual double calculateCharge(int days) = 0;
    virtual string getRoomType() = 0;
    virtual ~Room() {}
};
```

9.2 Derived Room Class

```
class DeluxeRoom : public Room {
public:
    double calculateCharge(int days) override {
        return days * 3000;
    }
```

```
    string getRoomType() override {
        return "Deluxe Room";
    }
};
```

9.3 Runtime Polymorphism

```
Room *r = new DeluxeRoom();

cout << r->getRoomType() << endl;
cout << r->calculateCharge(3) << endl;

delete r;
```

The complete source code should be included in the final GitHub submission.

10. ENVIRONMENT AND TOOLS USED

Component	Technology
Programming Language	C++
Programming Paradigm	Object-Oriented Programming
IDE	Visual Studio Code / Code::Blocks
Compiler	GCC / MinGW
Version Control	Git
Repository	GitHub
Documentation	Microsoft Word
Operating System	Windows

11. DEBUGGING APPROACH

Debugging was performed to identify syntax, logical and runtime errors.

Error	Cause	Solution
Compilation error	Incorrect class declaration	Correct class syntax
Ambiguous inheritance	Multiple copies of base class	Use virtual inheritance
Incorrect output	Wrong calculation	Correct billing logic
Room double booking	Availability not checked	Validate room status
Runtime crash	Invalid pointer operation	Correct pointer management
Polymorphism failure	Missing virtual function	Declare function as virtual
Memory leak	Dynamically allocated object not deleted	Use delete
Incorrect overriding	Function signature mismatch	Use override

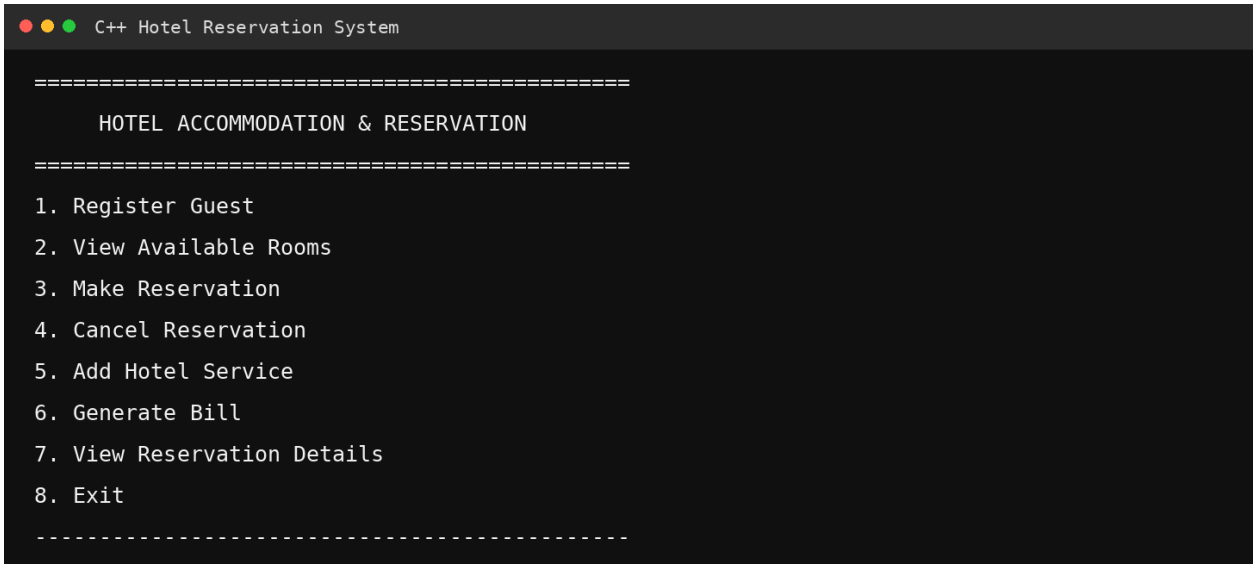
12. TEST CASES AND RESULTS

Test Case	Input	Expected Result	Actual Result	Status
TC01	Register valid guest	Guest registered	Guest registered	Pass
TC02	Select available room	Room can be reserved	Room reserved	Pass
TC03	Select occupied room	Reservation rejected	Reservation rejected	Pass
TC04	Cancel reservation	Room becomes available	Room released	Pass
TC05	Add food service	Service charge added	Charge added	Pass

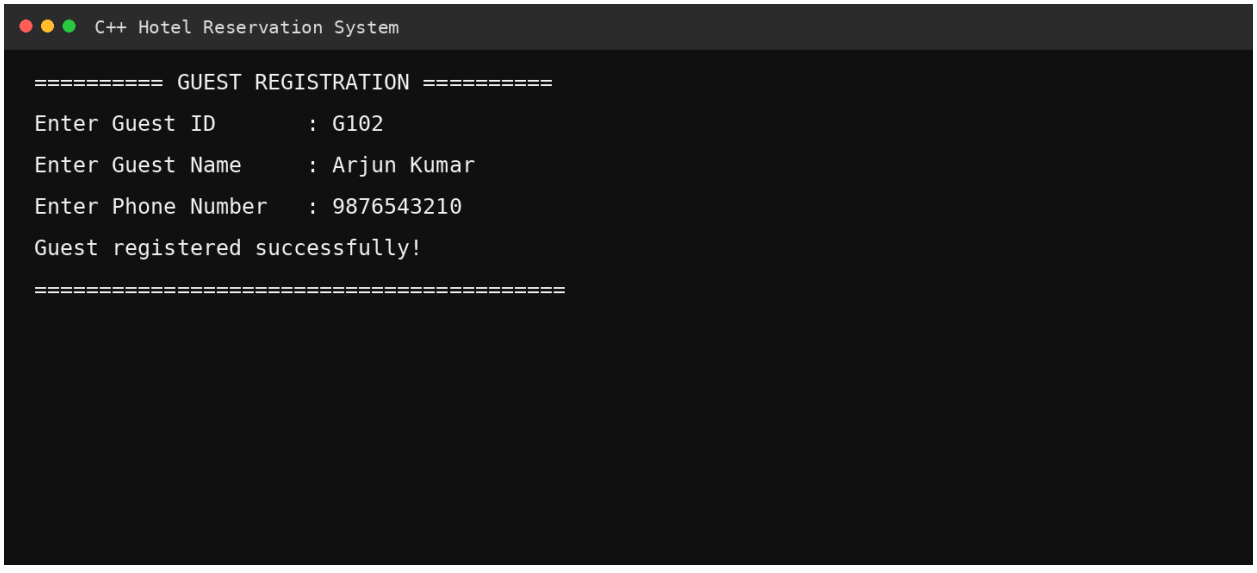
TC06	Generate bill	Correct total displayed	Correct total displayed	Pass
TC07	Invalid menu choice	Error message displayed	Error message displayed	Pass
TC08	Invalid room number	Invalid room message	Message displayed	Pass

13. EXECUTION SCREENSHOTS AND OUTPUTS

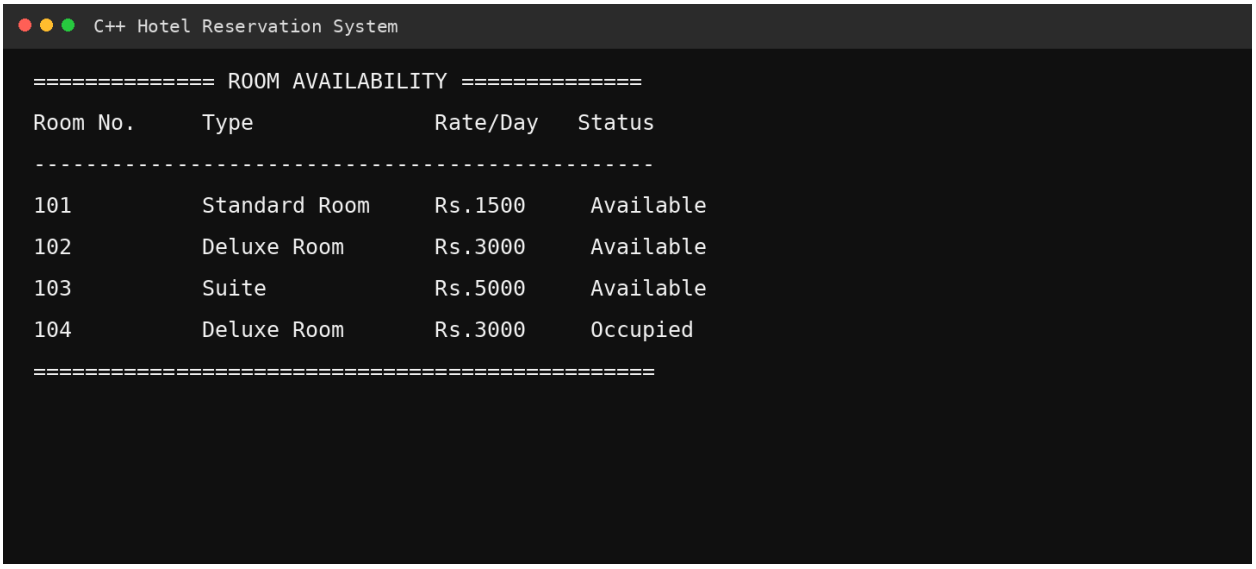
Screenshot 1 – Main Menu



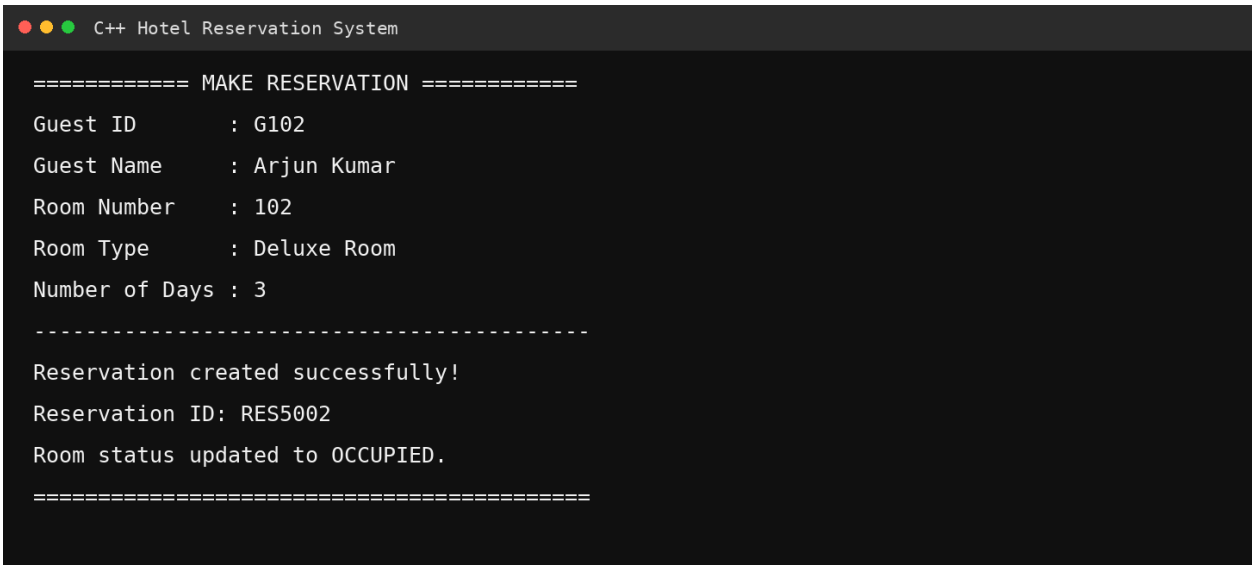
Screenshot 2 – Guest Registration



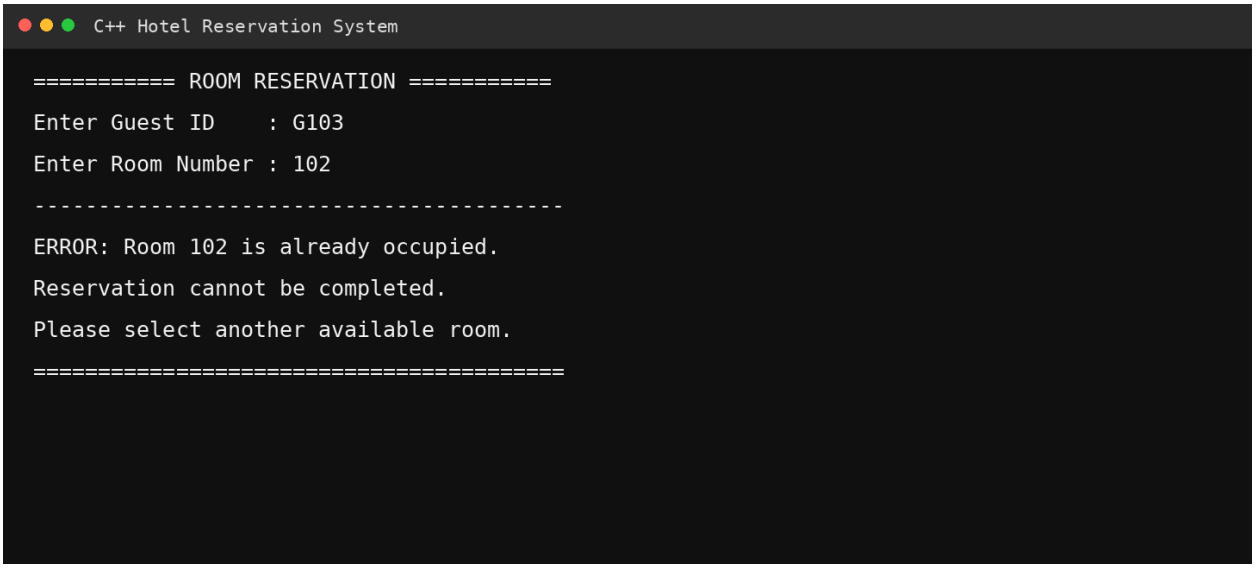
Screenshot 3 – Available Rooms



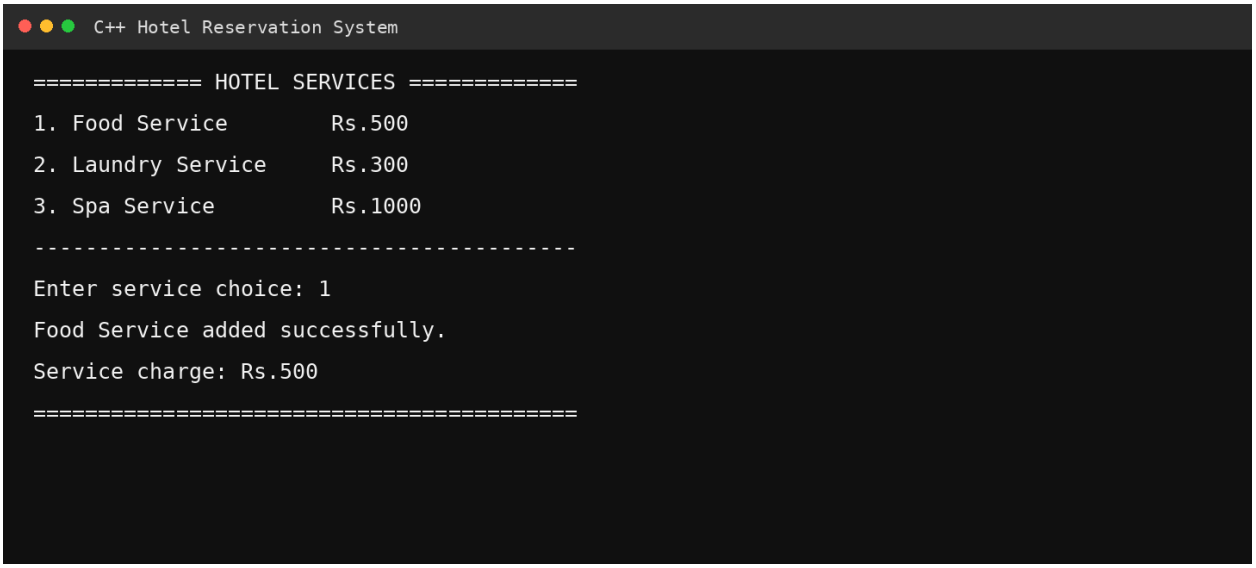
Screenshot 4 – Successful Reservation



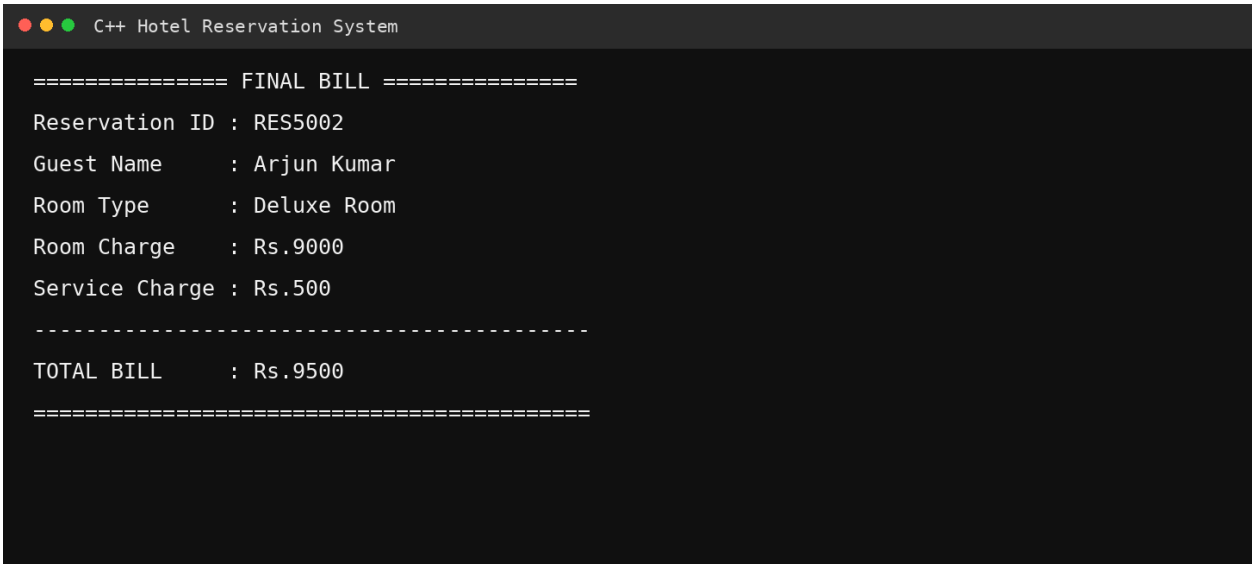
Screenshot 5 – Occupied Room Validation



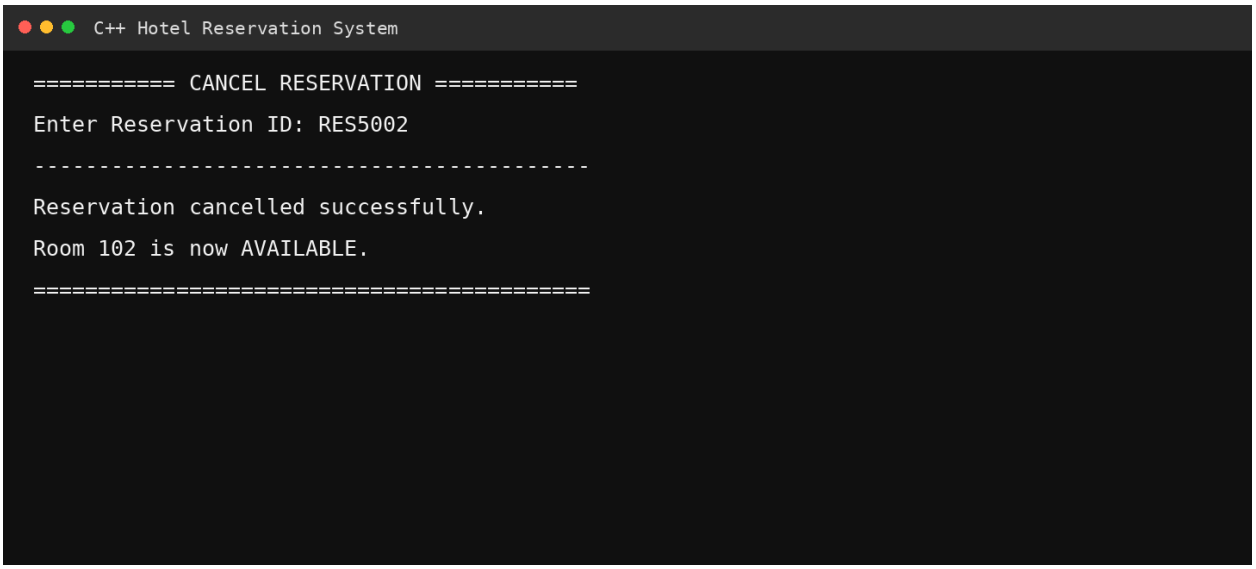
Screenshot 6 – Hotel Service Selection



Screenshot 7 – Bill Generation



Screenshot 8 – Reservation Cancellation



14. RESULTS AND VALIDATION

The developed system successfully demonstrates hotel accommodation and reservation operations through a menu-driven C++ application. The testing process verified guest registration, room availability, reservation creation, duplicate booking prevention, cancellation, service addition, bill calculation, runtime polymorphism and invalid-input handling.

The test results demonstrate that the proposed system satisfies the major functional requirements.

15. ANALYSIS, COMPARISON AND JUSTIFICATION

A simple procedural implementation could manage hotel reservations using functions and arrays. However, such an approach becomes difficult to maintain when new room types or services are added. The object-oriented approach provides better modularity, reuse, abstraction and extensibility.

Feature	Procedural Approach	Proposed OOP Approach
Code Reusability	Low	High
Maintainability	Moderate	High
Extensibility	Low	High
Inheritance	Not available	Available
Polymorphism	Not directly supported	Supported
Data Encapsulation	Limited	Strong
New Room Types	Requires major changes	New derived class
Debugging	More difficult	Modular debugging

Therefore, the OOP approach is more suitable for the proposed hotel management application.

16. BROADER CONSIDERATIONS / SDG RELEVANCE

The project has relevance to United Nations Sustainable Development Goal 9 – Industry, Innovation and Infrastructure. The system demonstrates the use of software technology to digitize hotel accommodation and reservation operations. Automation can reduce manual record keeping, improve operational efficiency and provide a structured approach to managing hospitality services.

- Digital transformation
- Efficient resource management
- Reduction of manual errors
- Technology-driven service management
- Scalable software design

17. INDIVIDUAL CONTRIBUTION

Member	Contribution
Member 1	Designed class hierarchy, room management and inheritance implementation.
Member 2	Developed guest registration and reservation management.
Member 3	Developed service management, billing, debugging and testing.

All members participated in integration, debugging, documentation and final validation.

18. INDIVIDUAL REFLECTION

Working on the Hotel Accommodation and Reservation Management System using C++ was a useful experience for me because it helped me understand how the concepts learned in class can be applied to a real-world application. I was involved in the development of the system and also helped in connecting different parts of the project, such as guest registration, room management, reservations, hotel services, billing, and debugging.

While designing the project, we decided to use different classes for the major functions of the hotel system. This made the program easier to understand and manage. We used inheritance to create different types of rooms and services. We also used abstract classes and pure virtual functions to define common features for different room and service types. Runtime polymorphism was implemented using pointers, which allowed different types of room objects to be handled through a common base class. We also used virtual inheritance to avoid duplication when multiple inheritance was involved.

During the development process, I faced a few difficulties. One of the main challenges was understanding how the different classes should be connected without causing ambiguity. I also had some issues with function overriding, pointers, room availability checking, and calculating the bill correctly. Finding and fixing these errors helped me understand debugging better. I learned that even a small mistake in a condition, function, or pointer can affect the overall output of the program. Testing the program with different inputs helped us identify and correct these problems.

This assignment helped me improve my understanding of important C++ concepts such as classes and objects, encapsulation, inheritance, abstraction, virtual functions, pure virtual functions, pointers, runtime polymorphism, constructors, destructors, and virtual base classes. More importantly, I learned how these concepts work together in a complete application rather than studying them only as separate topics.

The project is also related to Sustainable Development Goal 9 – Industry, Innovation and Infrastructure. A hotel reservation system can reduce manual work and make hotel operations more organized and efficient. Using software for reservations, room management, services, and billing shows how technology can be used to improve everyday business activities.

Overall, this assignment helped me improve my C++ programming, debugging, and problem-solving skills. It also helped me understand the importance of proper planning, testing, and teamwork when developing a software application. I feel that this project gave me more confidence in using Object-Oriented Programming concepts and contributed to achieving the mapped Course Outcome (CO) by helping me design, implement, debug, and evaluate a practical C++ solution.

.

19. CONCLUSION

The Hotel Accommodation and Reservation Management System successfully demonstrates the implementation of a real-world application using C++ Object-Oriented Programming concepts. The system provides facilities for guest registration, room availability, reservation management, cancellation, hotel services and bill calculation.

The use of abstract classes, inheritance, virtual base classes, pointers and runtime polymorphism improves the modularity and extensibility of the application. Debugging and validation helped identify and correct syntax, logical and runtime errors.

The project demonstrates that an object-oriented approach is suitable for developing maintainable and reusable hotel management software.

20. LIMITATIONS AND FUTURE IMPROVEMENTS

20.1 Limitations

- Data is not permanently stored.
- The application uses a console interface.
- No online payment facility is included.
- Multiple users cannot access the system simultaneously.
- No external database is connected.

20.2 Future Improvements

- Integrating MySQL or another database.
- Developing a graphical user interface.
- Adding online payment functionality.
- Implementing user authentication.
- Adding email/SMS reservation notifications.
- Developing an administrator dashboard.
- Adding customer feedback and rating features.
- Creating a web or mobile version.

21. REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
2. R. Lafore, *Object-Oriented Programming in C++*, 4th ed. Indianapolis, IN, USA: Sams Publishing, 2002.
3. S. B. Lippman, J. Lajoie, and B. E. Moo, *C++ Primer*, 5th ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2012.
4. P. Deitel and H. Deitel, *C++ How to Program*, 10th ed. Boston, MA, USA: Pearson, 2017.
5. International Organization for Standardization, *Information Technology—Programming Languages—C++*, ISO/IEC 14882. Geneva, Switzerland: ISO, 2020.